

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

ĐỒ ÁN MÔN TRÍ TUỆ NHÂN TẠO

Nghiên cứu và trình bày phương pháp phân loại văn bản bằng Multinomial Naive Bayes

Lớp: **D23VHCN01-N**

Lại Huy Thịnh — N23DVCN057

Nguyễn Trần Mạnh Dũng — N23DVCN01

Nguyễn Hữu Đức — N23DVCN012

Giảng viên:

Ngày hoàn thiện: 01/10/2026

Mục lục

1. Bài toán
2. Biến văn bản thành đặc trưng
3. Nguyên lý Multinomial Naive Bayes
4. Ví dụ tính tay
5. Thiết kế thực nghiệm
6. Kết quả thực nghiệm và tối ưu hóa siêu tham số
7. Phân tích lỗi chi tiết (Error Analysis)
8. Triển khai ứng dụng Web tương tác (Streamlit)
9. Kiểm thử tự động với pytest
10. Tài liệu tham khảo

1. Bài toán

Phân loại văn bản là gán một nhãn trong tập lớp đã biết cho một tài liệu mới. Trong đề tài này, đầu vào là nội dung một bài đăng tiếng Anh; đầu ra là một trong bốn chủ đề: đồ họa máy tính (comp.graphics), bóng chày (rec.sport.baseball), không gian (sci.space) hoặc chính trị tổng hợp (talk.politics.misc). Nhóm sử dụng bộ dữ liệu 20 Newsgroups qua `fetch_20newsgroups` của `scikit-learn` [1].

Mục tiêu là giải thích và kiểm chứng **một thuật toán phân loại**, Multinomial Naive Bayes (MNB). Bag of Words và TF-IDF là hai cách biểu diễn đầu vào cho cùng thuật toán đó. Bốn lớp và tập dữ liệu tiếng Anh là phạm vi thực nghiệm; chưa thể suy kết quả trực tiếp cho tiếng Việt.

2. Biến văn bản thành đặc trưng

Máy học nhận véc-tơ số. Với Bag of Words, mỗi chiều ứng với một từ trong bộ từ vựng; giá trị là số lần từ xuất hiện trong tài liệu. Ví dụ bộ từ vựng [bóng, đá, máy] biến câu “bóng đá bóng” thành [2, 1, 0]. Cách này giữ tần suất nhưng không giữ thứ tự từ.

TF-IDF giảm trọng số của từ xuất hiện trong nhiều tài liệu và tăng tương đối trọng số của từ mang tính phân biệt. `TfidfVectorizer` của `scikit-learn` kết hợp bước đếm từ với biến đổi TF-IDF [2]. MNB được xây dựng cho đặc trưng đếm không âm; `scikit-learn` ghi nhận véc-tơ TF-IDF cũng có thể dùng được trong thực hành [3]. Vì vậy, thí nghiệm so sánh `CountVectorizer` với `TfidfVectorizer` trong khi giữ nguyên thuật toán MNB.

Với cả hai cách, bộ từ vựng và trọng số IDF chỉ được học từ tập huấn luyện. Tập kiểm tra chỉ đi qua phép biến đổi đã học, tránh rò rỉ thông tin.

3. Nguyên lý Multinomial Naive Bayes

Theo định lý Bayes, với tài liệu d và lớp c :

$$P(c | d) = P(d | c) P(c) / P(d).$$

Vì $P(d)$ như nhau cho mọi lớp khi xét cùng một tài liệu, ta chọn lớp có giá trị $P(d | c) P(c)$ lớn nhất. MNB xem tài liệu như các lần xuất hiện của đặc trưng và áp dụng giả định độc lập có điều kiện: khi đã biết lớp, đóng góp của từng đặc trưng được tính riêng. Đây là phép xấp xỉ để mô hình đơn giản và dễ huấn luyện; các từ trong ngôn ngữ thực tế vẫn có quan hệ với nhau.

Với véc-tơ $x = (x_1, \dots, x_V)$, điểm dự đoán được tính ở miền log:

$$\text{score}(c, x) = \log P(c) + \sum_i x_i \log \theta(c, i).$$

Trong đó V là kích thước bộ từ vựng; x_i là số lần xuất hiện hoặc trọng số không âm của đặc trưng i ; $\theta(c, i)$ là xác suất đặc trưng đó trong lớp c . Miền log giúp tránh nhân nhiều số rất nhỏ. Xác suất tiên nghiệm $P(c)$ thường ước lượng từ tỷ lệ tài liệu thuộc lớp c trong tập train.

Ước lượng có làm trơn cộng α :

$$\theta(c, i) = [N(c, i) + \alpha] / [N(c) + \alpha V], \text{ với } N(c) = \sum_i N(c, i).$$

$N(c, i)$ là tổng giá trị đặc trưng i ở các tài liệu train thuộc lớp c . Khi $\alpha = 1$, đây là làm trơn Laplace. Nó giữ xác suất khác 0 cho từ chưa từng thấy trong một lớp [3]. Trong thực nghiệm, $\alpha = 1$ được cố định trước khi xem tập test.

4. Ví dụ tính tay

Giả sử có bốn câu huấn luyện, tách theo dấu cách:

Lớp	Câu huấn luyện
Thể thao	“bóng đá thắng”
Thể thao	“bóng đá đội”
Công nghệ	“máy tính nhanh”
Công nghệ	“máy tính mới”

Bộ từ vựng có 8 từ: bóng, đá, thắng, đội, máy, tính, nhanh, mới. Mỗi lớp có 2 trong 4 câu nên $P(\text{Thể thao}) = P(\text{Công nghệ}) = 1/2$. Mỗi lớp có 6 lượt từ. Với $\alpha = 1$, mẫu số cho mỗi xác suất từ là $6 + 8 = 14$.

Xét câu cần phân loại “**bóng đá mới**”:

Từ	$P(\text{từ} \mid \text{Thể thao})$	$P(\text{từ} \mid \text{Công nghệ})$
bóng	3/14	1/14
đá	3/14	1/14
mới	1/14	2/14

Điểm chưa chuẩn hóa của Thể thao là $(1/2) \times (3/14) \times (3/14) \times (1/14) = 9/5488$. Điểm của Công nghệ là $(1/2) \times (1/14) \times (1/14) \times (2/14) = 2/5488$. Vì $9 > 2$, mô hình dự đoán **Thể thao**. Ví dụ này minh họa cơ chế, không phải số liệu thực nghiệm trên 20 Newsgroups.

5. Thiết kế thực nghiệm

- Dùng bốn lớp đã nêu và đúng hai tập train/test do bộ dữ liệu cung cấp [1].
- Gọi `remove=('headers', 'footers', 'quotes')` ở cả train và test. Metadata có thể tiết lộ lớp; bộ lọc này giảm rủi ro nhưng không loại bỏ hoàn toàn [4].
- Huấn luyện hai pipeline: `CountVectorizer + MultinomialNB` và `TfidfVectorizer + MultinomialNB`, cùng $\alpha = 1$.
- Đánh giá accuracy toàn bộ, precision/recall/F1 theo lớp, macro F1 và ma trận nhầm lẫn. Macro F1 cho mỗi lớp trọng số như nhau, hữu ích khi số mẫu từng lớp khác nhau [5].
- Lưu số mẫu, phiên bản thư viện, tham số, kết quả và một số trường hợp sai được rút từ dự đoán trên test. Không dùng test để chọn tham số.

6. Kết quả thực nghiệm và tối ưu hóa siêu tham số

6.1. So sánh cơ sở giữa Bag of Words và TF-IDF ($\alpha = 1.0$)

Chạy `src/run_experiment.py` với scikit-learn cho **2.239** tài liệu train và **1.490** tài liệu test:

Biểu diễn đầu vào	Accuracy	Macro F1	Số đặc trưng
Bag of Words (CountVectorizer)	0,8544	0,8520	13.068
TF-IDF (TfidfVectorizer)	0,8718	0,8687	13.068

6.2. Tối ưu hóa siêu tham số Alpha bằng Stratified 5-Fold Cross-Validation

Để cải thiện độ chính xác mà **hoàn toàn không rò rỉ dữ liệu test**, nhóm thực hiện tìm kiếm siêu tham số làm trơn Laplace/Lidstone alpha trên tập dữ liệu train thông qua Stratified 5-Fold Cross-Validation (src/tune_alpha.py). Mỗi fold đều độc lập fit TfidfVectorizer trên 4 fold huấn luyện và transform trên fold kiểm thực tế.

Giá trị alpha	CV Accuracy trung bình	CV Macro F1 trung bình
0.01	0,8982 (+/- 0,0076)	0,8977 (+/- 0,0082)
0.05	0,9026 (+/- 0,0090)	0,9023 (+/- 0,0095)
0.10	0,9026 (+/- 0,0102)	0,9028 (+/- 0,0107)
0.20	0,9017 (+/- 0,0086)	0,9018 (+/- 0,0091)
0.50	0,8973 (+/- 0,0040)	0,8973 (+/- 0,0037)
1.00 (mặc định)	0,8861 (+/- 0,0051)	0,8855 (+/- 0,0049)
1.50	0,8740 (+/- 0,0106)	0,8725 (+/- 0,0115)
2.00	0,8593 (+/- 0,0123)	0,8553 (+/- 0,0135)

Dựa trên điểm CV Macro F1 cao nhất trên tập train, **alpha = 0.1** được chọn và khóa lại để huấn luyện mô hình cuối cùng trên toàn bộ 2.239 mẫu train.

6.3. Đánh giá mô hình tối ưu trên tập kiểm thử Test

Đánh giá đúng MỘT LẦN trên 1.490 mẫu test độc lập (results/evaluation_summary.json):

Chỉ số	Trước tối ưu (alpha=1.0)	Sau tối ưu (alpha=0.1)	Mức tăng (điểm phần trăm)
Test Accuracy	0,8718 (87,18%)	0,8852 (88,52%)	+1,34
Macro Precision	0,8763 (87,63%)	0,8841 (88,41%)	+0,78
Weighted Precision	0,8742 (87,42%)	0,8857 (88,57%)	+1,15
Macro Recall	0,8657 (86,57%)	0,8835 (88,35%)	+1,78
Weighted Recall	0,8718 (87,18%)	0,8852 (88,52%)	+1,34
Macro F1-Score	0,8687 (86,87%)	0,8833 (88,33%)	+1,46
Weighted F1-Score	0,8709 (87,09%)	0,8849 (88,49%)	+1,40

Chi tiết từng lớp với alpha=0.1:

- comp.graphics: Precision 0,9297 | Recall 0,9177 | F1 0,9237 (389 mẫu)
- rec.sport.baseball: Precision 0,8685 | Recall 0,9320 | F1 0,8991 (397 mẫu)
- sci.space: Precision 0,8865 | Recall 0,8325 | F1 0,8586 (394 mẫu)
- talk.politics.misc: Precision 0,8516 | Recall 0,8516 | F1 0,8516 (310 mẫu)

7. Phân tích lỗi chi tiết (Error Analysis)

Dựa trên kết quả chạy thực tế trên 1.490 mẫu test (results/error_analysis.json):

- **Dự đoán đúng:** 1.319 mẫu (88,52%).
- **Dự đoán sai:** 171 mẫu (11,48%).

7.1. Các cặp lớp thường nhầm lẫn nhất

1. sci.space -> rec.sport.baseball: 25 lần; sci.space -> talk.politics.misc: 25 lần.
2. talk.politics.misc -> sci.space: 23 lần; talk.politics.misc -> rec.sport.baseball: 19 lần.
3. sci.space -> comp.graphics: 16 lần.

Các số đếm được lấy từ ma trận nhầm lẫn của mô hình cuối. Chúng cho biết lớp nào bị nhầm, chưa chứng minh nguyên nhân. Văn bản pha trộn chủ đề hoặc thiếu từ phân biệt là những khả năng cần kiểm tra bằng từng mẫu sai cụ thể.

7.2. Tác động của văn bản ngắn / thiếu từ vựng

Trong 1.490 mẫu test, có **60 mẫu** có ít hơn hoặc bằng 2 từ vựng nằm trong bộ từ vựng TF-IDF đã học (chủ yếu do bước lọc bỏ header, footer, quote khiến văn bản chỉ còn vài từ ngắn hoặc rỗng).

- Tỷ lệ lỗi ở nhóm ít từ vựng là **53,33%** (32/60 mẫu sai), cao hơn gấp 5 lần so với nhóm bình thường (**9,72%**). Điều này khẳng định MNB phụ thuộc mật thiết vào tần suất từ khóa mang tính phân biệt chủ đề.

7.3. Phân tích độ tin cậy thấp (Confidence < 60%)

- Có **266 mẫu** có xác suất dự đoán cao nhất nhỏ hơn 60%. Tỷ lệ lỗi trong nhóm này lên đến **44,36%** (118/266 mẫu sai).
- Trong khi đó, ở nhóm có độ tin cậy từ 60% trở lên (1.224 mẫu), tỷ lệ lỗi chỉ là **4,33%** (53/1.224 mẫu). Nhờ đó, ứng dụng có thể sử dụng ngưỡng 60% làm chỉ báo cảnh báo người dùng khi độ tin cậy chưa cao.

8. Triển khai ứng dụng Web tương tác (Streamlit)

Hệ thống được chuẩn hóa kiến trúc hướng dịch vụ (**Service-Oriented Architecture**) và thiết kế giao diện đa mục đích phục vụ bảo vệ đề tài:

- **Tách tầng dịch vụ (src/classifier_service.py)**: Lớp TextClassifierService độc lập đảm nhiệm nạp mô hình, tiền xử lý an toàn (xử lý an toàn văn bản rỗng, None, từ vựng ngoài từ điển OOV), trích xuất TF-IDF, tính xác suất và tính toán mức đóng góp log-odds của từng từ khóa đối với lớp dự đoán. app.py chỉ tập trung xử lý giao diện người dùng.

- **Tổ chức 4 Tab chuyên biệt phục vụ thuyết trình**:

1. **Tab 1 - Dự đoán & Giải thích**: Ô nhập văn bản, phân loại thời gian thực, đo độ trễ xử lý (ms), bảng và biểu đồ xác suất 4 lớp, bảng giải thích mức đóng góp của các từ khóa TF-IDF, cảnh báo độ tin cậy thấp và xuất lịch sử phiên ra file CSV.
 2. **Tab 2 - Giới thiệu mô hình**: Trình bày trực quan quy trình pipeline Text → Preprocessing → TF-IDF → MultinomialNB → Prediction, công thức định lý Bayes, log-space và nguyên lý làm trơn Laplace.
 3. **Tab 3 - Đánh giá thực nghiệm**: Nạp động kết quả từ results/evaluation_summary.json và results/alpha_tuning.json; hiển thị các metric Test Accuracy (88,52%), Macro F1 (88,33%), Classification Report, Confusion Matrix và bảng so sánh 5-Fold Cross-Validation của 8 giá trị alpha.
 4. **Tab 4 - Giới hạn & Lưu ý**: Phân tích các giới hạn phương pháp luận về tập dữ liệu tiếng Anh, từ ngữ OOV, văn bản quá ngắn và bản chất xác suất hậu nghiệm của Naive Bayes.
- **Độc kết quả thực nghiệm động**: Không hard-code chỉ số; toàn bộ số liệu hiển thị trên giao diện được đọc tự động từ các tệp kết quả với cơ chế xử lý lỗi an toàn.

9. Kiểm thử tự động với pytest

Dự án có **27 ca kiểm thử** trong test_pipeline.py, test_inference.py, test_ui_behavior.py và test_release_consistency.py. Các nhóm kiểm tra gồm:

1. Nạp mô hình MultinomialNB thành công từ file .joblib.
2. Nạp bộ véc-tơ TF-IDF thành công (13.068 đặc trưng).
3. Dự đoán trả về nhãn hợp lệ trong 4 chủ đề.
4. Xử lý an toàn chuỗi rỗng (""), khoảng trắng (" ") và giá trị None (tham số hóa 4 ca kiểm thử).
5. Văn bản chứa từ ngữ ngoài từ điển (OOV) không gây lỗi crash ứng dụng và kích hoạt fallback/cảnh báo an toàn.

6. Xác suất `predict_proba` nằm trong khoảng hợp lệ `[0, 1]`.
7. Tổng xác suất các lớp xấp xỉ 1.0.
8. Mô hình nhận diện chính xác 4 lớp của bài toán.
9. Khả năng dự đoán chính xác văn bản mới chưa từng xuất hiện.
10. Kiểm tra cảnh báo giao diện UX khi gặp đầu vào bất thường (văn bản ngắn, OOV, độ tin cậy thấp).
11. Trích xuất giải thích đặc trưng TF-IDF tiêu biểu kèm mức độ ủng hộ cho lớp dự đoán.
12. Xác thực tính toàn vẹn của các file artifact mô hình (`test_inference.py`).
13. Đảm bảo độ chính xác trên các mẫu suy diễn thực tế chưa từng thấy (`test_inference.py`).

Ngoài suy diễn, bộ kiểm thử hành vi dùng Streamlit AppTest xác minh đầu vào rỗng không tạo lượt dự đoán, khoảng trắng không tăng lịch sử, đoạn trích đúng ở biên 80/81 ký tự và dữ liệu lịch sử xuất CSV.

Lệnh thực thi: `python -m pytest -q`; lần hậu kiểm ngày 01/10/2026 đạt **27/27**. Đây là số ca kiểm thử phần mềm đạt, không phải độ chính xác 100% của mô hình.

Kết luận

TF-IDF kết hợp Multinomial Naive Bayes đạt Accuracy 88,52% và Macro F1 88,33% trên 1.490 mẫu test. Phương pháp phù hợp để minh họa phân loại văn bản có giám sát và cơ chế tính điểm xác suất. Đồ án giới hạn ở văn bản tiếng Anh thuộc bốn chủ đề; chưa được đánh giá cho tiếng Việt hoặc chủ đề ngoài tập lớp. Xác suất dự đoán chưa được hiệu chỉnh, vì vậy cần xem cảnh báo và kiểm tra kết quả khi đầu vào ít thông tin.

10. Tài liệu tham khảo

[1] scikit-learn, `fetch_20newsgroups`.

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_20newsgroups.html

[2] scikit-learn, `TfidfVectorizer`.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[3] scikit-learn, Naive Bayes, mục Multinomial Naive Bayes.

https://scikit-learn.org/stable/modules/naive_bayes.html

[4] scikit-learn, Classification of text documents using sparse features.

https://scikit-learn.org/stable/auto_examples/text/plot_document_classification_20newsgroups.html

[5] scikit-learn, `classification_report`.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html